Ricerca predittiva regolata

Un esperimento sul giro di consegne

Francesco Vampa
Matricola: in attesa

22 ottobre 2026

Indice

1 Introduzione **2**

1.1 Complessità computazionale 2

2 Algoritmi di ricerca **2**

2.1 Rappresentazione dello stato e spazio di ricerca 2

2.2 Espansione e generazione dei successori 3

2.3 Ordine di visita tramite DFS 3

2.4 Eliminazione dei rami tramite branch-and-bound 4

2.5 Orizzonte, frontiera attiva e condizioni di arresto 4

Riferimenti bibliografici **5**

1 Introduzione

Il problema del giro di consegne consiste nel determinare un percorso che permetta a un veicolo di partire da un deposito, visitare tutti i punti di consegna una sola volta e tornare al deposito, minimizzando la distanza totale percorsa. In questo lavoro, i luoghi sono rappresentati da punti su un piano e gli spostamenti sono valutati mediante distanze euclidee. Il problema corrisponde quindi al problema del commesso viaggiatore euclideo (*Euclidean Traveling Salesman Problem*, TSP euclideo).

Il numero dei possibili ordini di visita cresce rapidamente all'aumentare delle consegne, rendendo utile scegliere quali alternative esplorare quando il calcolo disponibile è limitato. L'obiettivo del progetto è valutare se un modello predittivo possa guidare la ricerca verso percorsi promettenti e se un regolatore possa migliorarne ulteriormente l'efficacia, decidendo dove dedicare ulteriore calcolo. A questo scopo, verranno confrontate una ricerca di base, una ricerca predittiva e una ricerca predittiva regolata, utilizzando budget di ricerca comparabili.

1.1 Complessità computazionale

Il TSP è un problema di ottimizzazione **NP-hard**, anche quando le distanze sono euclidee [1]. La classificazione riguarda la famiglia di istanze di dimensione variabile: non implica che ogni singola istanza sia difficile da risolvere. In particolare, istanze piccole possono essere risolte esattamente e utilizzate come riferimento negli esperimenti.

Per il TSP generale, la NP-hardness può essere dimostrata mediante una riduzione polinomiale dal problema del circuito hamiltoniano [2]. Dato un grafo $G = (V, E)$, si costruisce un grafo completo sugli stessi vertici, assegnando costo 0 agli archi presenti in E e costo 1 agli archi aggiunti. Il grafo originale contiene un circuito hamiltoniano se e solo se il grafo costruito ammette un giro TSP di costo 0. Un algoritmo esatto per il TSP permetterebbe quindi di decidere l'esistenza del circuito hamiltoniano. Quest'ultimo visita ogni *vertice* una sola volta, mentre un circuito euleriano attraversa ogni *arco* una sola volta.

Questa costruzione riguarda il TSP generale: i costi così assegnati non sono necessariamente rappresentabili come distanze euclidee. La NP-hardness della variante euclidea è un risultato distinto, richiamato in [1]. Nel presente progetto si mantiene questa variante e si studia la qualità delle soluzioni ottenute con risorse di ricerca limitate, senza pretendere di eliminare la difficoltà del problema nel caso generale.

2 Algoritmi di ricerca

In questa sezione viene descritto l'algoritmo di ricerca adottato per risolvere il problema del giro del corriere. In particolare, l'algoritmo utilizza una visita in profondità (DFS) per determinare l'ordine di visita dei nodi, con l'aggiunta di un meccanismo di branch-and-bound che ci consente di scartare i rami che non possono portare a una soluzione migliore di quella già trovata.

2.1 Rappresentazione dello stato e spazio di ricerca

Prima di descrivere l'algoritmo, si definiscono i concetti di stato e spazio di ricerca.

Uno *stato* rappresenta una configurazione del problema in un dato momento della ricerca. Nel problema del giro del corriere, ciascuno stato contiene il percorso parziale costruito fino a quel momento, l'insieme delle consegne ancora da effettuare e la distanza accumulata. La posizione attuale del corriere corrisponde all'ultimo punto del percorso.

Lo stato iniziale contiene il solo deposito nel percorso, tutte le consegne ancora da effettuare e un costo pari a zero. Quando tutte le consegne sono state visitate, il giro viene completato tornando al deposito. La distanza di ritorno viene inclusa nel costo finale.

Lo *spazio di ricerca* è l'insieme degli stati raggiungibili dallo stato iniziale attraverso le azioni consentite. In questo caso, comprende tutti i percorsi parziali ammissibili e i giri completi. I percorsi iniziano dal deposito e non ripetono alcun punto di consegna; i giri completi visitano tutte le consegne e terminano nuovamente al deposito.

2.2 Espansione e generazione dei successori

L'espansione di uno stato consiste nella generazione dei possibili proseguimenti del percorso corrente.

Sia $V = \{d, c_1, \ldots, c_n\}$ l'insieme delle località, dove d rappresenta il deposito e $c_1, \ldots, c_n$ i punti di consegna. Per ciascuno stato si distinguono l'insieme $C \subseteq V$ delle località già visitate, comprendente il deposito, e l'insieme $N = V \setminus C$ delle consegne ancora da effettuare. Si ha quindi:

$$C \cap N = \emptyset, \qquad C \cup N = V.$$

L'ordine delle località visitate è conservato separatamente nel percorso p, mentre g indica il costo accumulato.

Per ogni consegna $v \in N$ viene generato uno stato successore. Il nuovo percorso si ottiene aggiungendo v alla fine di p, mentre gli insiemi vengono aggiornati come:

$$C' = C \cup \{v\}, \qquad N' = N \setminus \{v\}.$$

Indicando con u l'ultima località del percorso corrente, il costo del successore è:

$$g' = g + \mathrm{dist}(u, v).$$

2.3 Ordine di visita tramite DFS

Per determinare l'ordine di esplorazione degli stati, l'algoritmo utilizza una **visita in profondità** (DFS, *Depth-First Search*), implementata mediante una pila con politica LIFO: l'ultimo stato inserito è il primo a essere estratto. La procedura è riportata nell'Algoritmo 1.

Algoritmo 1 Depth-First Search

Dati: Stato iniziale s_0
Risultato: Visita degli stati in profondità

```
S ← pila vuota                                  ▷ Conserva gli stati in attesa
push(S, s_0)
while S non è vuota do
    s ← pop(S)                                  ▷ Estrae l'ultimo inserito
    visita s
    L ← Successori(s)                           ▷ Genera le estensioni ammissibili
    Ordina L per indice crescente della consegna aggiunta
    for s' ∈ L in ordine inverso do             ▷ Esplora prima gli indici minori
        push(S, s')
    end for
end while
```

La funzione Successori(s) applica la regola di espansione definita nella sottosezione precedente. Se non rimangono consegne da effettuare, restituisce una lista vuota. I successori vengono inseriti nella pila in ordine decrescente di indice, così che quello con indice minore venga estratto per primo. La visita termina quando la pila è vuota.

La DFS determina l'ordine di esplorazione, ma non riduce di per sé il numero di stati da esaminare. In assenza di potatura, il numero complessivo di stati attraversati è $\Theta(n!)$, dove n indica il numero di consegne, escluso il deposito. Questa crescita fattoriale limita la scalabilità della ricerca esaustiva e motiva l'introduzione del branch-and-bound, che consente di eliminare i rami che non possono migliorare la migliore soluzione completa già trovata.

2.4 Eliminazione dei rami tramite branch-and-bound

2.5 Orizzonte, frontiera attiva e condizioni di arresto

Riferimenti bibliografici

[1] Mark de Berg et al. *An ETH-Tight Exact Algorithm for Euclidean TSP*. 2018. arXiv: 1807.06933. URL: https://arxiv.org/abs/1807.06933 (visitato il 22/10/2026).

[2] OpenDSA. *Reduction of Hamiltonian Cycle to Traveling Salesman*. Materiale didattico CS201, Duke University, primavera 2025. 2025. URL: https://opendsa.cs.vt.edu/ODSA/Books/duke/cs201/spring-2025/MW/html/hamiltonianCycle_to_TSP.html (visitato il 22/10/2026).